

Assignment 4

JavaScript Alerts and Confirms – Selenium Automation Lab Report

Name	Roshni Ghosh Tagore
Enrollment No.	12023002028175
Course	Python for Automation
Academic Year	2023-2027
Department	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus	IEM Newtown

Module: Advanced User Interactions (Selenium WebDriver)

Experiment Title: Handling JavaScript Alerts, Confirm Boxes, and Prompt Boxes

1. Problem Statement

Modern web applications frequently use native JavaScript pop-ups such as alert boxes, confirmation boxes, and prompt boxes to communicate messages or collect quick input from the user. Since these pop-ups are rendered outside the normal HTML DOM, standard Selenium element-locating commands cannot interact with them directly. The task is to automate a practice web page that triggers a JS Alert, a JS Confirm box, and a JS Prompt box, and to correctly detect, read, and respond to each of them using Selenium WebDriver.

2. Objective

- To trigger a JavaScript Alert, Confirm box, and Prompt box on a practice web page.
- To capture and print the message text displayed inside each pop-up using `driver.switch_to.alert`.
- To accept the JS Alert box.
- To dismiss the JS Confirm box.
- To enter text into the JS Prompt box using `send_keys()` and accept it.

3. Tools, Software, and Concepts Used

Category	Details
Programming Language	Python 3
Automation Framework	Selenium WebDriver
Browser / Driver	Google Chrome (ChromeDriver)
IDE	Visual Studio Code
Practice Website	the-internet.herokuapp.com/javascript_alerts
Key Concepts	<code>driver.switch_to.alert</code> , <code>alert.accept()</code> , <code>alert.dismiss()</code> , <code>alert.send_keys()</code> , <code>By.XPATH</code> locators, <code>time.sleep()</code> for wait handling

4. Source Code / Implementation Steps

The script below launches Chrome, navigates to the practice page, and handles the three types of JavaScript pop-ups in sequence: Alert → Confirm → Prompt.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/javascript_alerts")
driver.maximize_window()

# --- JS Alert ---
driver.find_element(
    By.XPATH, "//button[text()='Click for JS Alert']"
).click()

alert = driver.switch_to.alert
print("Alert message:", alert.text)
time.sleep(3)
alert.accept()

# --- JS Confirm ---
driver.find_element(
```

```

        By.XPATH, "//button[text()='Click for JS Confirm']"
    ).click()

    alert = driver.switch_to.alert
    print("Confirm message:", alert.text)
    time.sleep(3)
    alert.dismiss()

    # --- JS Prompt ---
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Prompt']"
    ).click()

    alert = driver.switch_to.alert
    print("Prompt message:", alert.text)
    time.sleep(3)

    alert.send_keys("Hello Selenium")
    time.sleep(2)
    alert.accept()
    print("Prompt accepted successfully")
    time.sleep(2)

    driver.quit()

```

Implementation steps: (1) Launch the Chrome browser and open the target URL. (2) Click the 'Click for JS Alert' button and switch control to the alert using `driver.switch_to.alert`, then read its text and accept it. (3) Click the 'Click for JS Confirm' button, read the confirm box text, and dismiss it (Cancel). (4) Click the 'Click for JS Prompt' button, read the prompt text, type input using `alert.send_keys()`, and accept the prompt to submit the text.

5. Output with Screenshots

Screenshot 1: Implementation code (Part 1) – Alert and Confirm handling

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/javascript_alerts")
driver.maximize_window()

driver.find_element(
    By.XPATH, "//button[text()='Click for JS Alert']"
).click()

alert = driver.switch_to.alert

print("Alert message:", alert.text)

time.sleep(3)

alert.accept()

driver.find_element(
    By.XPATH, "//button[text()='Click for JS Confirm']"
).click()

alert = driver.switch_to.alert

print("Confirm message:", alert.text)

time.sleep(3)

alert.dismiss()

```

Fig. 1: Python/Selenium script handling the JS Alert (accept) and JS Confirm (dismiss).

Screenshot 2: Implementation code (Part 2) – Prompt handling

```

driver.find_element(
    By.XPATH, "//button[text()='Click for JS Prompt']"
).click()

alert = driver.switch_to.alert

print("Prompt message:", alert.text)

time.sleep(3)

alert.send_keys("Hello Selenium")

time.sleep(2)

alert.accept()

print("Prompt accepted successfully")

time.sleep(2)

driver.quit()

```

Fig. 2: Python/Selenium script entering text into the JS Prompt and accepting it.

Screenshot 3: Console output

```

sktop\ALL_SELENIUM\ScreenShots\assignment4.py"
Alert message: I am a JS Alert
Confirm message: I am a JS Confirm
Prompt message: I am a JS prompt
Prompt accepted successfully

```

Fig. 3: Terminal output showing the captured Alert, Confirm, and Prompt messages, and confirmation that the prompt was accepted successfully.

6. Result, Observation, and Conclusion

Result:

The script executed successfully and produced the following console output, confirming that all three JavaScript pop-ups were correctly detected and handled:

```

Alert message: I am a JS Alert
Confirm message: I am a JS Confirm
Prompt message: I am a JS prompt
Prompt accepted successfully

```

Observation:

- Native JavaScript pop-ups are not part of the HTML DOM, so they cannot be located with usual By locators; `driver.switch_to.alert` must be used to bring them into Selenium's control.
- `alert.accept()` clicks the 'OK' button, while `alert.dismiss()` clicks 'Cancel' — this was verified by dismissing the Confirm box instead of accepting it.
- `alert.send_keys()` successfully injected text into the Prompt box's input field before it was accepted with `alert.accept()`.
- Adding short `time.sleep()` pauses made it possible to visually verify each pop-up state before it was handled.

Conclusion:

The experiment successfully demonstrated Selenium WebDriver's ability to handle the three standard types of JavaScript pop-ups – Alert, Confirm, and Prompt – using the `switch_to.alert` interface along with `accept()`, `dismiss()`, and `send_keys()`. This forms a foundational skill for automating real-world web applications that rely on native browser dialogs for notifications and user input.